

Building a Production-Grade Agentic Database Assistant

Memory Handling Best Practices

Design the assistant's memory in layered tiers to mimic human recall ¹. A **short-term memory** buffer holds the most recent dialogue turns (e.g. last 5–10 exchanges) to provide immediate context ². This acts like a sliding window so the LLM always sees recent queries and answers without overflowing its context. An **episodic memory** summarization captures key themes, entities, and user preferences over the session – essentially a running session summary ³. For example, the system can track “active topics” or important **entities mentioned** in the conversation ⁴, so even if earlier turns drop out of the short-term window, their gist is retained. Finally, **long-term memory** persists across sessions by saving conversation logs and insights to disk (e.g. JSON files) so the assistant can recall facts or past resolutions from previous sessions ⁵ ⁶. This multi-tier memory (recent turns + session summary + stored long-term context) prevents the model from forgetting important details while avoiding an overload of irrelevant text.

To implement this, integrate the memory system with the assistant's observability and logging. Each turn saved to memory should produce a structured log event for traceability ⁷. For example, after the assistant generates a response, it creates a `ConversationTurn` object and saves it via the `ConversationMemory.add_turn` method, then logs an operation `"memory.save_turn"` with the session and turn ID ⁷. This ensures every memory update is auditable, linking to a trace ID and session ID in the logs ⁸ ⁹. By unifying memory with observability, developers can replay and inspect what the assistant “knew” at each step, which is crucial for debugging and compliance auditing.

Checkpointing and pruning: Regularly condense and trim the conversation history to prevent context overflow and hallucination. For example, the system might summarize older turns into an episodic memory entry every N turns or when a topic changes, then remove those turns from the active window. The code already prunes the short-term list to the last 10 turns ², and it could be extended by generating a summary of those turns (via the LLM) before pruning. This checkpointing ensures the prompt stays focused on relevant information and mitigates the risk of the LLM picking up and elaborating on outdated or irrelevant details (a common source of hallucinations). Additionally, memory management should drop or compress less useful details – e.g. after resolving a sub-question or when switching context, the assistant can retain only key conclusions in the summary and not carry over the entire dialogue. Overall, a layered, **pruned memory** approach keeps the context window lean and pertinent, much like a human analyst who remembers recent dialogue verbatim but only the high-level takeaways from earlier discussion.

Auto-Traversal for Any Question

Equip the agent with the ability to automatically expand vague or incomplete questions by iterative **query understanding and context gathering**. When a user asks an open-ended question (e.g. “*How are our customers doing?*”), the assistant should **detect key entities or concepts** mentioned and use them to dig

deeper. The intent classification step can extract potential table names, column names, or business terms from the query ¹⁰. For example, simple rules identify tokens like `PIO_` prefixes or dot notation as table references, known business terms like “customer” or “AML” as domain entities, and technical words like “schema” or “column” as hints the user is asking about structure ¹⁰. These entities serve as starting points for traversal.

Next, the assistant performs a **hybrid schema search** to retrieve any schema elements or documentation relevant to the query. A combination of BM25 keyword search and embedding-based semantic search ensures high recall ¹¹. In practice, the system might query a **catalog of tables/columns** for names or descriptions matching the user’s terms (lexical search) while also using vector similarity to catch semantic matches (e.g. “client” $\approx$ “customer”) ¹¹. The results from both methods are merged and re-ranked for relevance. In addition, a graph search is used to expand the context: the schema is represented as a graph of tables, columns, and their relationships, so the assistant can find not only direct matches but also **neighboring entities**. For instance, if a user mentions a table, the graph can suggest connected tables via foreign keys. The system’s `hybrid_search` explicitly blends semantic results with graph-based related entities, boosting entries that are connected in the schema graph ¹² ¹³. This way, even if the user’s question is vague, the agent gathers a network of relevant schema objects – ensuring it “misses nothing” important.

Using these techniques, the assistant can **recursively traverse** the domain. It might start with an initial entity (say “customer”), retrieve candidate tables (e.g. Customer master table, Account table, Transaction table related to customers), then for each candidate, fetch their schema details and even join paths between them. The `ProductionSchemaRetriever` can find join paths between tables using graph algorithms ¹⁴. This is crucial for complex questions that involve multiple entities – for example, “*What is the average transaction amount per high-risk customer?*” requires joining a Customers table with a Transactions table and perhaps a Risk rating table. The agent can automatically discover such connections (via foreign key relationships in the graph) and determine how to compose an SQL query across them ¹⁴.

Once the relevant schema context is gathered, the assistant **enriches the LLM prompt** with this information before attempting to answer or generate SQL. Rather than only providing a few vector search snippets, it gives the LLM a holistic picture: e.g. “*Table CUSTOMER (contains customer profile info, with primary key customer_id) links to ACCOUNT via customer_id. Table TRANSACTION stores transactions with fields amount, account_id (foreign key to ACCOUNT). High-risk customers are flagged in CUSTOMER.risk_score.*” This additional context empowers the LLM to formulate correct and join-aware SQL queries and to answer in a way that aligns with the database structure.

Moreover, the agent should handle follow-up questions and clarifications by carrying over context automatically. The router can detect follow-up phrases like “which one of those is latest?” or “what about those columns?” and infer the reference from recent dialogue ¹⁵. For example, if the user’s last question was about certain columns of a table, and now they ask “how do those columns relate to X?”, the system knows “those columns” refers to the ones mentioned earlier. Internally, the intent classifier will tag the new query as a `FOLLOW_UP` intent with the previously identified entities attached ¹⁵. The agent then **persists recent entities in context** (via `context_carryover`) so that the next tool invocation knows to include those in its search or SQL generation. This mechanism enables a seamless conversational flow: the user can ask incremental questions without restating all details, and the agent will automatically traverse the conversation state to fill in the blanks. In summary, **auto-traversal** involves detecting query components,

expanding them through schema and graph searches, and using conversation context to maintain continuity – all to answer even vague questions with relevant, specific data.

Edge Case Handling

A robust production system must anticipate and gracefully handle a variety of edge cases and failure modes:

- **Missing Vector Index or Embeddings:** If the semantic search infrastructure is unavailable or not yet initialized (e.g. embedding model not loaded or vector database down), the agent should fall back to an alternative retrieval method. The system logs a warning in such cases (e.g. "BGE embeddings not available, using fallback" ¹⁶) and can revert to using a simpler **catalog text search** or keyword-based lookup ¹¹ . This ensures the assistant can still fetch relevant schema info or documentation even if the vector search fails. In addition, if the vector store is present but an embedding **dimension mismatch** is detected (for instance, stored vectors are 384-dimensional but the model now produces 768-dimensional vectors), the startup process flags it as a critical issue ¹⁷ . In production, you'd want to automatically handle this by possibly re-embedding the data or ignoring the mismatched collections. In the provided code, the system lists such mismatches and even suggests running a fix script ¹⁸ . Until the issue is resolved, the assistant should avoid using the affected embeddings (to prevent erroneous similarity results) and rely on other data sources.
- **Empty or Ambiguous Query Results:** If a user's query yields no relevant hits (for example, no tables or documents match the query terms), the assistant should neither crash nor hallucinate an answer. Instead, it can respond with a **constructive fallback message**. A good practice is to explain that no information was found and guide the user on how to refine the query. The system's `_generate_fallback_response` does exactly this – it returns a friendly note like *"I couldn't find specific information about 'X'. This could mean it doesn't exist or the question is too broad..."* and then offers suggestions for related topics or keywords to try ¹⁹ . This template-driven answer gives the user actionable next steps. If partial context is available, the assistant might also provide **contextual hints**; for example, noticing the word "customer" and suggesting *"Try asking about customer data or KYC information"* ²⁰ ²¹ . By handling empty results with helpful prompts, the system remains engaging rather than just saying "I don't know."
- **Ambiguous Intent or Classification:** Sometimes the user's intent isn't clear – e.g. a question could be interpreted as either a schema question or a data query. In such cases, the router should use a cautious default or ask a clarifying question. The design here leans on an LLM-powered intent classifier to catch nuances, but if that fails, a rule-based fallback classifies uncertain queries as a general data question by default ²² (since it's safer to assume a database question and use tools than to miss a data request). For truly ambiguous inputs, another strategy is to have the answer composer politely ask the user for clarification (e.g. *"I'm not sure what you need. Do you want information about the database schema, or actual data from it?"*). This kind of clarification step can be integrated as another node in the agent's decision graph if confidence is below a threshold.
- **Abusive or Irrelevant Queries:** The assistant should include **safety guards** to handle profanity, harassment, or off-topic requests. The router's intent classifier can detect abuse by matching common toxic phrases ²³ and label the intent as `ABUSE` . When such intent is detected, the system should avoid normal tool usage and instead produce a brief, neutral response that deflects the

abuse and reminds the user of the assistant's purpose ²⁴ . For instance, the SafetyGuards class defines a set of polite responses like *"I focus on database and data analysis tasks. How can I assist you with your AML system?"* ²⁵ . The agent can randomly choose one of these canned replies to address the user. This keeps the interaction professional and on-track. In a production setting, additional measures like rate limiting abusive users (to prevent spam or denial-of-service attempts) can be applied – the scaffold for rate limiting is present (e.g. a placeholder method in SafetyGuards) ²⁶ .

- **SQL Errors and Timeouts:** When executing generated SQL against the database, things can go wrong – the query might time out, or the database might throw an error (syntax error, missing table, etc.). The system's design accounts for this by including a dedicated error-handling node in the agent workflow (the `_handle_error_node`). If the SQL executor fails or any exception bubbles up, the orchestrator diverts to the error handler which can invoke the LLM to **generate an informative error message** for the user ²⁷ . For example, if the answer composer fails to format the results or the SQL query didn't run, the agent might respond with: *"I'm sorry, something went wrong while retrieving the data. Would you like me to try a different approach?"* ²⁸ . This is preferable to a raw stack trace or a silent failure. Additionally, the agent can implement retries or self-healing: for a timeout, maybe try a simpler query (e.g. limit results, or sample the data) and inform the user that a partial answer is given due to performance issues. All such errors should be logged via observability with details ²⁹ ³⁰ , so engineers can later analyze and address root causes (e.g. identify if a particular type of query frequently fails).
- **Missing Schema Metadata:** If the assistant cannot find schema information for a referenced entity (perhaps a new table isn't documented in the catalog), it should handle this gracefully. One approach is a fallback to a direct database inspection – for instance, the agent could run a quick query against the information schema or system tables to get the schema of the unknown table. If that's not possible (or not allowed due to permissions), the assistant can tell the user, *"I don't have information on the structure of table X"*, and possibly prompt if they want to run a schema-introspecting query. In the provided code, the SchemaRetriever checks if the schema index database exists and returns an error if not ³¹ , which the agent could convey to the user. Robust design would include a default path when metadata is missing: either query the live schema directly or present a fallback message rather than leaving the LLM to hallucinate the schema.
- **Incomplete Database Configuration (Oracle Config):** In a production scenario, the agent relies on a database connection (in this case to an Oracle DB) to execute SQL. If the connection details are missing or incorrect, the system should detect this at startup and restrict its functionality accordingly. The code attempts to load Oracle credentials from environment or config, and if not found, it logs a warning ³² and proceeds with an empty config (potentially using a local SQLite as a placeholder) ³³ ³⁴ . In practice, if no live DB is available, the agent should **disable or mock** the data querying capability – perhaps only answer schema questions from the knowledge base. It should inform the user if a data question is asked in this state, e.g. *"(Note: Live database access is currently not available, so I can only provide schema information.)"*. By handling this case, we avoid the agent generating SQL that it can never execute. Ideally, the system would have a health check at startup: no DB config means either fail to launch with an alert, or run in a read-only documentation mode. The provided system even instantiates a "legacy" agent as a fallback using whatever config is available ³⁵ , which is a pattern you can use: maintain a simpler answer path that doesn't rely on the full toolset for use in degraded mode.

In all these edge cases, a key principle is **graceful degradation**. The agent should never simply break; it should catch errors and either try an alternate method or return a helpful message. The answer composer and router include multiple fallbacks to implement this. For example, if the LLM fails to compose a final answer, the system falls back to a simple listing of top search results ³⁶. If that too fails, a final generic message is given. This tiered fallback ensures that the user always gets *something* relevant or at least an explanation, rather than silence or a confusing error. All such decisions are logged via the observability framework (with structured error logs and performance metrics), so operators can monitor how often fallbacks are needed and refine the system over time ³⁷ ³⁸.

Full Database Overview for the LLM

To empower the LLM with a comprehensive understanding of the database, the system should build and supply a **holistic schema overview**. This involves aggregating schema structure, metadata, and relationships into an accessible form:

- **Schema Mapping:** The assistant should maintain an internal representation of the database schema – essentially a map of *tables* → *columns* → *data types/meanings*. In practice, a component (like `SchemaRetriever` or a `CatalogStore`) loads the list of tables and their column info, including any descriptions or comments. This can be done by reading from a dedicated schema database or catalog. For example, the system uses an SQLite `schema_db_path` which presumably contains a snapshot of the AML database schema (possibly including table comments, column descriptions, etc.) ³⁹ ⁴⁰. All tables, along with column names and types, keys, and constraints, are extracted. Additionally, any **business metadata** is included: if the data dictionary provides a business purpose for each table or flags certain columns as sensitive or required by compliance, those are recorded as well. In the AML context, many tables/fields likely have AML-specific classifications (e.g. a flag indicating a column is required for regulatory reports). The system's data dictionary collection (e.g. `aml_dictionary_metadata`) would contain such info, which is made available to the agent.
- **Graph of Relationships:** Beyond individual tables, the assistant needs to know how tables relate to each other. This is achieved by constructing a graph of the schema where nodes are tables or columns and edges are foreign key relationships or conceptual links. The production system integrates with a Neo4j graph database that stores this schema graph ⁴¹. Using this, the agent can query for connections (like “find how Table A joins to Table B”) and get a path of join relationships ¹⁴. This is invaluable for answering questions that span multiple tables. For instance, if the user asks for something that involves accounts and transactions, the schema graph will reveal that the Accounts table links to Transactions via an `account_id`, and perhaps Accounts link to Customers via `customer_id`, etc. The assistant can surface those connections to the LLM so it understands the join path. In a compressed form, the assistant might tell the LLM: “*Accounts table is linked to Transactions table by account_id (foreign key).*” Including primary keys, foreign keys, and referential integrity info prevents the LLM from guessing relationships – it will know them exactly from context.
- **Dictionary Metadata and AML Flags:** The knowledge base should also incorporate the **purpose and classification** of each schema element. For example, a table might be annotated with “Business Purpose: Stores customer KYC (Know Your Customer) information” and columns might be tagged as “AML Required” or “Sensitive PII”. The AnswerComposer is designed to inject these into the answers – it takes `table_info` that includes total column count, how many are AML-required, and the table's purpose ⁴². It then explicitly asks the LLM to mention these in the final answer ⁴³. This

approach ensures the answers aren't just technically correct, but also **business-aware**. The user might not only want the raw data, but also an interpretation: e.g. instead of saying "Table X has 5 columns," the answer would say "Table X has 5 columns in total, 3 of which are required for AML compliance, and it is used for storing transaction alerts" ⁴³. This enrichment provides context that a domain expert would typically mention. The agent sources these details from the data dictionary and passes them along. In implementation, when the SchemaRetriever fetches a table's details, it likely retrieves fields like `business_description`, `data_classification`, or `AML_REQUIRED` flags (as seen in the metadata parsing in answer composer) ⁴⁴ ⁴⁵. All of this is compiled into a structured `table_info` object and then into a natural language prompt for the LLM.

- **Compressed Delivery to LLM:** Because the schema information can be very large (hundreds of tables, each with dozens of columns), it's neither feasible nor useful to always feed the entire schema to the LLM. Instead, the system should **summarize and contextualize on demand**. There are a few strategies to do this:
 - *On-demand retrieval:* Given a specific user query, pull only the schema fragments that are relevant to that query. For example, if the question mentions "customer" and "transactions," retrieve the Customer-related tables and Transaction-related tables (and maybe their immediate neighbors). The assistant can dynamically assemble a short description of each relevant table (name, purpose, key columns) to include in the prompt. This targeted schema context keeps the prompt efficient.
 - *Pre-summarized schema notes:* The team could maintain a pre-written summary of the entire database (or major sections of it) that the LLM can consult. For instance, a summary might say: "We have 5 main subject areas: Customers, Accounts, Transactions, Alerts, and Reference Data. Customers are stored in PIO_CUSTOMER table (with demographics and risk info), Accounts in PIO_ACCOUNT... Transactions in PIO_TXN... Alerts in AML_ALERTS..." etc. This high-level overview can be given at the start of a session or whenever the user asks a very broad question. It provides the LLM with a mental map of the database.
 - *Hierarchical detail:* The agent can decide how much detail to provide based on the query complexity. For a simple question about one table, just provide that table's description. For a question that potentially touches many tables (e.g. "give me an overview of all AML systems data"), provide a top-level summary and then ask the user to drill down if needed, or provide summaries of each section one by one.

The **AnswerComposer** component illustrates how schema context is injected in practice. It builds a textual context like: *"Table: TRANSACTION_LOG\nTotal columns: 12\nAML required columns: 5\nPurpose: Stores all executed transactions for monitoring.\n\nColumn details (8 shown): - txn_id (NUMBER): unique transaction identifier [AML required]\n- acct_id (NUMBER): account reference [AML required]\n- amount (DECIMAL): transaction amount in USD [AML required] ..."* and so on ⁴⁶ ⁴³. This is prepended to the LLM prompt so that the model's answer includes key structural info naturally. The model is instructed to mention the total column count, highlight AML-required fields, and explain the table's purpose in its answer ⁴³. By doing this for any table or set of tables in question, the assistant ensures the *LLM has a complete picture* of not just the names of things but their role and importance.

Finally, to make schema delivery efficient, implement **caching** for schema lookups. The schema doesn't change often, so caching results for retrievals can save time. The `SchemaRetriever` class in the system uses an in-memory cache with a TTL (e.g. 5 minutes) to store recent schema query results ⁴⁷ ⁴⁸. That means if the user asks multiple questions about the same table or term, the schema details are fetched

from cache on subsequent turns instead of hitting the database or index repeatedly. There is also likely a persistent cache (the `catalog.db` and `AI_AML.sqlite` files) that store the full schema and enable fast search. Upon startup, the system initializes these (and connects to Neo4j graph if configured) ⁴¹. By caching both in memory and on disk, the assistant achieves quick access to schema info with minimal latency during conversation. As a best practice, also **pre-warm** the cache: load the most important tables' info at startup, so the first question about them is answered without delay. The combination of summarization and caching allows the assistant to provide the LLM with rich schema context in a timely manner, which leads to more accurate and context-aware responses.

Standards & Industry Parallels

The described architecture aligns with several industry best practices and concepts in the AI and database assistant space:

- **Advanced Memory Mechanisms:** The memory layering we propose is analogous to approaches seen in frameworks like **LangChain** or **LangGraph** for conversational agents. For example, LangChain provides short-term conversation memory (a buffer window of recent interactions) and can be paired with long-term vector-based memory; our design implements a similar idea, with short-term and episodic memory in-memory and long-term stored externally. In fact, the system here uses a `ConversationMemory` that maintains recent turns and also stores an episodic summary and key entities ¹, which is very much in line with the state-of-the-art conversation memory modules. By structuring memory this way, the agent can handle follow-up questions and context switching more gracefully than a naive stateless system. These techniques are increasingly standard in industry chatbots to combat context window limits and maintain coherence over long sessions ⁴⁹.
- **Agentic Orchestration with Graphs:** The use of a **graph-based orchestrator** (e.g., the `StateGraph` in the code) is a modern engineering approach to building reliable AI agents. Rather than a linear script, the assistant's decision-making is modeled as a graph of states/nodes (classification → retrieve → generate SQL → validate → execute → answer compose → etc.), with conditional transitions and the ability to loop or retry on errors ⁵⁰ ⁵¹. This is similar to emerging "LangGraph" or workflow-based agent frameworks where each tool or step is a node in a directed graph. Such architectures are more maintainable and interpretable, letting the system do plan-act-observe-replan cycles systematically ⁵². Many industry solutions (including some using LangChain's agents or Microsoft's Guidance) are moving toward explicit state graphs for complex tool-using agents. Our design follows this trend, enabling features like the `_self_heal_node` for retrying failed actions and a clear separation of concerns for each stage.
- **Constitutional and Safe AI Practices:** To handle edge cases and ensure compliance, one can draw inspiration from **Anthropic's Constitutional AI** and similar safety frameworks. The idea is to bake in guidelines or "constitution" that the assistant must follow when generating responses, especially in sensitive domains like AML. For instance, rules like *"The assistant should never reveal confidential customer information"* or *"If asked to do something potentially illegal or against policy, the assistant should refuse"* can be enforced by an intermediate check. In practice, after the answer composer drafts a response, the system could invoke a secondary LLM pass that reviews the answer for safety or compliance issues and edits it if needed (this is analogous to how Constitutional AI has a critique and revision step). While the provided system uses a simpler approach (pattern matching for abuse

and preset refusals ²³ ²⁴), adding a more nuanced LLM-driven safety layer would align with industry best practices for AI governance. This ensures that even if the user tries to prompt the agent into giving sensitive details or doing something disallowed, the agent will consistently respond within the allowed policy.

- **Retrieval-Augmented Database Copilots:** The overall approach of combining retrieval (of schema/data dictionary info) with generation is following the **Retrieval-Augmented Generation (RAG)** paradigm, which is popular in many commercial applications now. Microsoft's Azure OpenAI service, for example, showcases use-cases where an LLM is hooked up to an **Azure SQL** database with an intermediary that feeds schema information and even executes SQL safely ⁵³ ⁵⁴. Our agent is essentially a specialized RAG pipeline for databases: it retrieves schema context and possibly sample data, then asks the LLM to compose answers or SQL, and finally executes the SQL to ground the answer. Similarly, companies like OpenAI and others have written about "SQL GPT" agents that use table metadata to help the model generate correct SQL queries. Amazon AWS's QuickSight Q and other BI tools also implement natural language to SQL by leveraging metadata and synonyms. The inclusion of a vector database for documentation and a graph for relationships in our design is an extra step that puts it at the cutting edge of these SQL copilots – it's not just using column names, but also the semantics and linkages, to drive understanding. This is very much in line with research directions where an agent builds an internal knowledge graph of the environment (here, the DB schema) to reason more effectively.
- **SQL Safety and Validation:** Ensuring the agent only runs safe, read-only queries is non-negotiable in production. The system explicitly validates each generated SQL against a set of rules before execution ⁵⁵ ⁵⁶. This is analogous to how, for example, OpenAI's Code Interpreter or other tools sandbox execution or validate inputs. By forbidding modification keywords (`INSERT`, `UPDATE`, `DROP`, etc.) and only permitting `SELECT` statements ⁵⁷, the assistant prevents any destructive actions on the database. It also checks for patterns like `--` or `UNION` that could indicate SQL injection or unwanted complexity ⁵⁸. Many industry solutions include such allow-deny lists; for instance, Microsoft's guidance for Azure SQL copilots also emphasizes wrapping the LLM's SQL output in a review step to ensure it's within a safe subset of SQL. Our agent's SafetyGuards fulfill this role, returning a structured result (`{"safe": True/False, "reason": ...}`) that the orchestrator can use to decide if execution should proceed ⁵⁵. Emulating **principle of least privilege**, the database user credentials provided to the agent should also be read-only at the DB level as an extra safety net (so even if a bad query slips through, it cannot do harm).
- **Structured Logging and Monitoring:** In enterprise settings, any AI agent interacting with critical data must have robust logging and monitoring. The observability module in our system produces **structured JSON logs** for every operation ⁸ ³⁷. This is aligned with industry standards like using ELK/Elastic Stack or Splunk to analyze logs, or employing OpenTelemetry for tracing AI workflows. Each user query can be traced end-to-end with a `trace_id`, linking the intent classification, tool calls, SQL execution, and final answer. Such traceability is invaluable not only for debugging but also for compliance – especially in AML, auditors may need to see why the assistant gave a certain answer or what data it accessed. We also track performance metrics (timings for each step) in the logs ⁵⁹ ⁶⁰, which parallels how production systems use APM (Application Performance Monitoring) tools to catch slow queries or bottlenecks. By following these logging practices, the assistant can be integrated into an enterprise observability framework from day one.

In summary, this agentic database assistant design brings together cutting-edge methods from conversational AI and practical safeguards from the software engineering world. It marries the dynamic reasoning of LLMs with a rich, structured understanding of the database, all while maintaining safety, compliance, and transparency. The result is a system that behaves like a savvy human data analyst: it remembers the conversation, explores the data schema autonomously to find answers, handles corner cases with poise, and explains data in business terms – but it never oversteps boundaries (like modifying data or violating policy). This approach is reflective of the latest **industry best practices** and should serve as a strong foundation for a production-grade AML database assistant.

Sources: The implementation details referenced above are drawn from the provided code and configuration of the system, including the memory management in `ConversationMemory` ^{1 2}, the routing and search logic in `ModernAgenticRouter` and `SchemaRetriever` ^{11 15}, safety and error handling in `SafetyGuards` and the orchestrator ^{55 27}, and the answer composition templates in `AnswerComposer` ^{43 46}. These illustrate the application of the best practices discussed.

^{1 2 3 4 6 10 15 22 23 24 25 26 55 56 57 58} `modern_agentic_router.py`

`file://file-NTCGvm9MfiSBzB13H9AK7T`

⁵ `agentic_behavior.py`

`file://file-1GKgVjibUpAMK7MqCB7UH4`

^{7 27 28 50 51 52} `modern_orchestrator.py`

`file://file-1nNZetAZvXvsQzCc3V2YHd`

^{8 9 29 30 37 38 59 60} `observability.py`

`file://file-LNuzEKkaShX5WyZ5RNUK33`

^{11 31 39 40 47 48} `schema_retriever.py`

`file://file-KaYEChbpaWhk6UCME7YXg`

^{12 13 14 16 41} `production_schema_retriever.py`

`file://file-7YQGyryeoAeVawbQns7d2D`

^{17 18 19 20 21 32 33 34 35} `aml_web_chat.py`

`file://file-QSRbVieUKzS4i9dpRK2AGE`

³⁶ `langgraph_orchestrator.py`

`file://file-K1mgZJwUr3GhRkzcKykUGs`

^{42 43 44 45 46} `answer_composer.py`

`file://file-RjWRWu8FN9gm7ZgEC6DZvL`

⁴⁹ `What Memory Type Do You Recommend? : r/LangChain - Reddit`

`https://www.reddit.com/r/LangChain/comments/173t400/what_memory_type_do_you_recommend/`

^{53 54} `Retrieval Augmented Generation with Azure SQL - Azure SQL Devs' Corner`

`https://devblogs.microsoft.com/azure-sql/rag-with-azure-sql/`